CAWASMA

Context-Aware Web & API Security Misconfiguration Analyzer

Final-year project — viva study notes

Asim Shakeel

Check specs, CVSS mapping, ML & evaluation

Roll number: 22P-9229

Supervisor: Dr. Bilal Khan

Team: Asim Shakeel (22P-9229)
Neelam Jabeen (22P-9353)
Atta Ur Rehman (22P-9236)

These notes are generated from the repository, documentation and git history.
Every number is cited to the file or command it came from. Date: November 10, 2026.

Contents

1 The whole project **2**
1.1 The problem . . . 2
1.2 The idea . . . 2
1.3 Architecture . . . 2
1.4 Technology, and why . . . 3
1.5 Team roles and the shared contracts . . . 3

2 What Asim built **4**
2.1 Check specifications and CVSS base vectors . . . 4
2.2 The environmental mapping (the core idea) . . . 4
2.3 Sensitivity classifier . . . 5
2.4 Response (body) classifier . . . 5
2.5 Chain templates . . . 5
2.6 The evaluation lab . . . 5

3 Numbers to know **6**
3.1 The mapping in action . . . 6
3.2 Classifier accuracy, honestly . . . 6
3.3 Detection and ranking . . . 6

4 Contribution timeline **7**

5 Viva preparation **7**
5.1 Likely questions, with short answers . . . 7
5.2 Three-minute demo script . . . 8
5.3 Five things to explain without notes . . . 9

6 Glossary **9**

1 The whole project

1.1 The problem

A normal security scanner gives a fixed severity to each kind of finding. A missing `X-Content-Type-Options` header is "medium" wherever it is found: on a public brochure page, and on a login API that returns credentials. The person reading the report then has to work out, by hand, which of a hundred "medium" findings actually matters. That triage is slow and easy to get wrong. Tools such as OWASP ZAP and Nuclei do find real issues, but they rate the issue, not the place it sits in.[1]

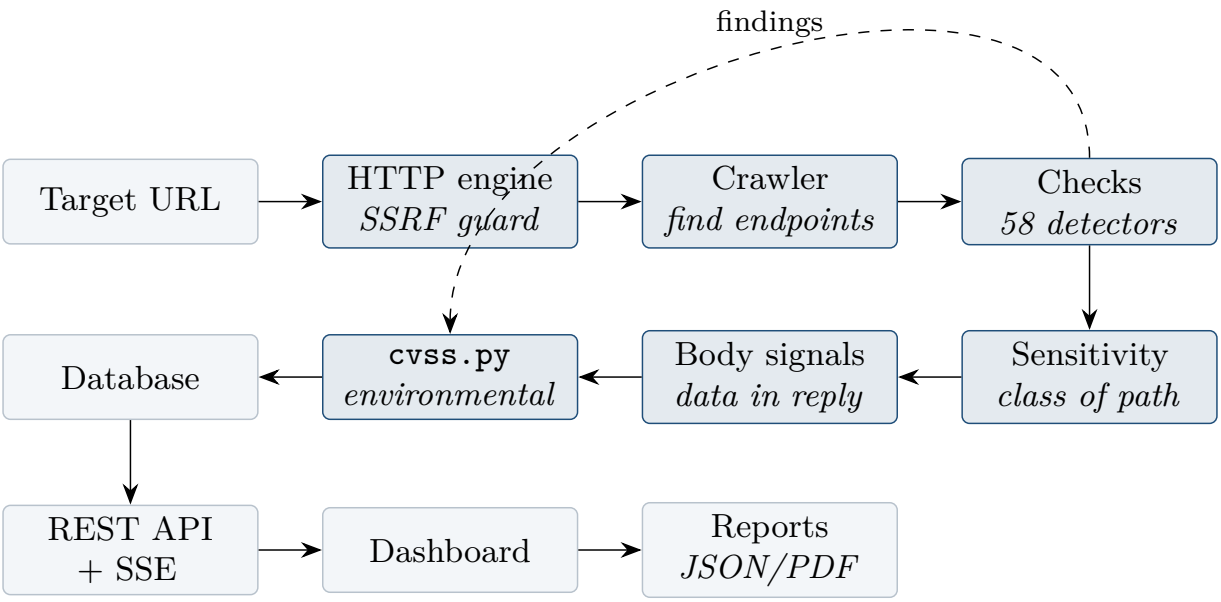
1.2 The idea

CAWASMA scores the *same* finding differently according to where it occurs and what the endpoint returned. It does this with the standard CVSS v3.1 *environmental* metrics, so the result is not an invented number: it is a recomputation of the official score with a vector anyone can check on the FIRST calculator.

One-paragraph pitch. CAWASMA is a defensive, black-box, HTTPS-only scanner for web apps and REST APIs. Each of its checks carries a fixed CVSS v3.1 base vector. For every finding it works out how sensitive the endpoint is (from the URL path) and what sensitive data the response holds (credentials, personal data, financial data, internal IPs). It turns those into CVSS environmental security requirements (CR/IR/AR) and recomputes the score, so a login API and a public page that share a weakness no longer share a priority. The point of the project is *prioritisation, not detection.*

1.3 Architecture

A scan is a pipeline. A URL goes in; a ranked, explained report comes out.



The stages are: the **engine** fetches safely (one SSRF check per redirect hop); the **crawler** discovers endpoints (seed, HTML, JavaScript, a wordlist, and OpenAPI paths); the **checks** look for misconfigurations; the **sensitivity** classifier labels each endpoint CRITICAL/HIGH/MEDIUM/LOW

[1] Source: `docs/supervisor/scanners.md`

from its path; the **body classifier** looks for sensitive data in the reply; `cvss.py` turns those two into CR/IR/AR and recomputes the score; the result is stored, served by the API over REST and Server-Sent Events, shown on the dashboard, and exported as reports.

1.4 Technology, and why

Part	Choice and reason
Scanner core	Python 3.12 with `httpx` + `asyncio`: endpoints are analysed concurrently, and the core stays database-free so it is easy to test.
Scoring	The `cvss` library, so the environmental maths is the published CVSS v3.1 formula, not ours.
NLP	Sentence-Transformers (`all-MiniLM-L6-v2`) and spaCy for the sensitivity class; the engine still works with the built-in classifier when these are absent.
API	FastAPI, with Pydantic schemas and a frozen contract; Celery + Redis run scans in the background with live status.
Database	PostgreSQL in production, SQLite in tests; the models stay portable with a `JSONType` column and database-level cascades.
TLS	SSLyze (with a plain-`ssl` fallback) for the transport checks.
Reports	One HTML template through WeasyPrint for the PDF, so HTML and PDF never drift.
Frontend	Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS v4, shadcn/ui, TanStack Query.
Tests	`pytest` (backend), Vitest + Testing Library and Playwright (frontend).

1.5 Team roles and the shared contracts

Three people own three areas, and they meet at three written contracts, frozen so one person's change cannot silently break another's code.[2]

Member	Owns
Asim (22P-9229)	Check specifications and their CVSS v3.1 vectors, the environmental mapping, chain templates, the sensitivity and response (ML/NLP) classifiers, and the whole evaluation lab.
Neelam (22P-9353)	The Next.js web app, the `frontend` service and its CI job.
Atta (22P-9236)	The backend: HTTP engine, crawler, check functions, CVSS scoring code, attack-chain engine, reports, FastAPI app, Celery workers, database and migrations, plus the shared infrastructure and reviews.

API contract (v1). `app/api/schemas/contract.py` and `docs/api-contract.md`: the field names the frontend reads (`check_id`, `base_score`, `final_score`, status `DONE`).

ML / CVSS contract. `app/core/contracts.py` and `docs/cvss-mapping-contract.md`: the types the engine and Asim's classifiers exchange, so the engine keeps working with the built-in classifiers when the ML packages are missing.

Check-spec format. `app/core/specs.py` and `docs/check-spec-format.md`: how a check is declared so the registry can run it.

Two scoring states. Not every check is re-scored the new way yet. Of 62 check ids, 25 have an approved specification and are scored through the CVSS v3.1 environmental metrics; the other 37 still use an interim formula, $\min(10,\ \text{base} \times \text{context multiplier} + \text{response bonus})$, and each finding records which method scored it (`scoring_method`).[3]

[2] Source: `CLAUDE.md; .github/CODEOWNERS`

[3] Source: `make coverage-stats; docs/06-scoring-model.md`

2 What Asim built

Asim owns the parts that decide *how severe* a finding is and the lab that measures whether the tool works: the check specifications and their CVSS v3.1 base vectors, the mapping from context to CR/IR/AR, the chain templates, the two classifiers (sensitivity and response), and the whole evaluation harness.[4]

2.1 Check specifications and CVSS base vectors

What it does. Every check carries a fixed CVSS v3.1 base vector and base score, declared once in `backend/app/core/checks/catalog.py` and, for the newer spec-driven checks, in `config/checks.yaml`. These are the official scores; they are never invented per scan.

How it works. Each catalog entry is a `RuleMeta` with the check id, title, base vector and references. The spec format (`docs/check-spec-format.md`) lets a check say what it `applies_to` and which `artifact` it reads, so the registry can run it. Of 62 check ids, 25 have an approved spec.

Why a fixed base vector per check? So the "before" number in every comparison is the published CVSS base score, not something the project chose. Only the *context* part is CAWASMA's contribution, and it is applied on top of this fixed base.

Inputs/outputs. Out: a base vector consumed by Atta's `cvss.py`. **Tested by** `make validate-config` (every vector parses and matches `checks.yaml`) and the catalog count guard. **Limitation:** 31 legacy entries have a catalog `base_score` that disagrees with their vector; the vector is treated as the source of truth.

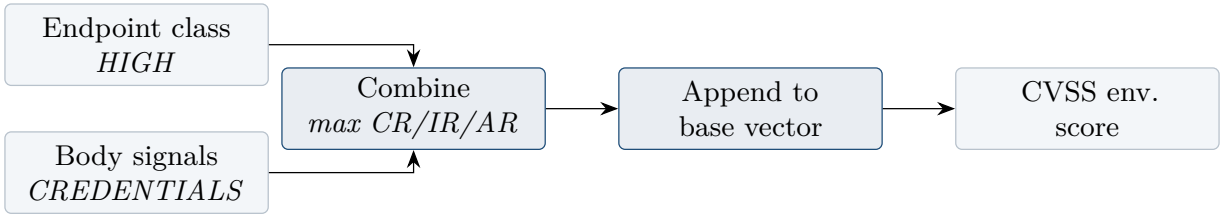
2.2 The environmental mapping (the core idea)

What it does. `config/cvss_mapping.yaml` turns an endpoint's sensitivity class and the data signals in its response into the CVSS environmental security requirements CR, IR and AR.

How it works. Each sensitivity class and each body signal has a CR/IR/AR triple. When several apply, the highest requirement wins (`combine: max`). A LOW (public) endpoint maps to X/X/X — Not Defined — which is the neutral baseline, so a public page keeps its base score. The result is appended to the base vector and recomputed by the `cvss` library.

Listing 1: The context-to-requirements mapping (`config/cvss_mapping.yaml`).

```
sensitivity_classes:
  LOW: {CR: X, IR: X, AR: X} # public: base stands
  MEDIUM: {CR: M, IR: M, AR: M}
  HIGH: {CR: H, IR: M, AR: M} # login/token/session
  CRITICAL: {CR: H, IR: H, AR: H} # admin/payment
body_signals:
  CREDENTIALS: {CR: H, IR: H, AR: M}
  PII: {CR: H, IR: M, AR: X}
  FINANCIAL: {CR: H, IR: H, AR: M}
  INTERNAL_IP: {CR: M, IR: X, AR: X}
combine: max # highest of class and all signals wins
```



[4] Source: `.github/CODEOWNERS; CLAUDE.md`

Design decisions. Using CVSS's own CR/IR/AR (not a custom weight) keeps the output a standard, reproducible score. "Max" is deliberately conservative: one strong reason to care (say, credentials in the body) is enough to raise the requirement, rather than averaging it away. X for LOW means context never *lowers* a public finding below its base. **Tested by** `make validate-config` and `tests/unit/test_cvss.py`. **Limitation:** the triples are expert judgement, defined once; they are not learned from data.

2.3 Sensitivity classifier

What it does. `backend/app/ml/sensitivity.py` labels an endpoint CRITICAL/HIGH/MEDIUM/LOW from the words in its path, so `cvss.py` knows which class triple to use.

How it works. `classify(endpoint)` tokenises the path (`path_text()`), optionally expands abbreviations, and compares it with class "anchors" using sentence-embedding similarity (`AnchorClassifier`, config in `config/sensitivity_anchors.yaml`). Static assets short-circuit to LOW. When the ML packages or the anchor config are absent, the engine falls back to Atta's built-in keyword classifier in `core/sensitivity.py`.

Design decision. The semantic anchor classifier ships *off by default*; the keyword families are the default path. The anchor set scores better on the evaluation (see Part C) but was kept optional to avoid a heavy dependency in the default install. **Tested by** `tests/ml/` and the sensitivity evaluation. **Limitation:** accuracy on a four-class problem is modest (about 0.49 on the test split); the class boundaries are genuinely fuzzy.

2.4 Response (body) classifier

What it does. `backend/app/ml/body_classifier.py` looks in a response for sensitive data and returns a set of signals: CREDENTIALS, PII, FINANCIAL, INTERNAL_IP. Evidence is always redacted.

How it works. `analyze(response)` runs `detect()`, which scans text (API-doc example blocks are stripped first) for credentials and secret keys, personal data, financial numbers (validated with Luhn / IBAN checks), and internal IPs. `shannon_entropy()` and `_random_looking()` help tell a real secret from a placeholder; `mask()` redacts a value to its last few characters before it is stored.

Design decision. The classifier is conservative: placeholder and example values are filtered so a documentation sample does not raise a real finding. **Tested by** `tests/ml/` (fixtures of real-looking and placeholder data). **Limitation:** it reads server responses only, so secrets rendered by client-side JavaScript are not seen.

2.5 Chain templates

What it does. `config/chains.yaml` declares the five attack-chain templates that Atta's `core/chains.py` correlates and scores.

How it works. Each template names the member checks, the scope they must share, and a composite CVSS vector. The five are: session theft via XSS; error disclosure plus version banner; credentialed CORS read on a sensitive API; credential interception on login; and exposed configuration leaking live credentials. **Tested by** `tests/unit/test_chains.py`. **Limitation:** a chain forms only where every member fires on the same scope; on the evaluation targets chains appear only on DVWA and crAPI.

2.6 The evaluation lab

What it does. `eval/` is how the project proves itself: real targets, a hand-written answer key, and a harness that scans with CAWASMA, ZAP and Nuclei and scores all three.

How it works.

- `eval/ground_truth/` holds, per target, which check should (`present`) or should not (`absent`) fire on which endpoint: 33 present, 20 absent and 27 `must_not_fire` entries.
- `harness/run_all.py` resets the four targets, puts an HTTPS proxy in front (so HTTPS-only CAWASMA can reach them), and scans with each tool.
- `harness/evaluate.py` normalises each tool's output to CAWASMA check ids (`tool_mapping.yaml`), matches it to the answer key by target, check and endpoint, and computes precision, recall and FPR. A tool is scored only on the checks its rules can produce.
- `harness/ranking.py` computes NDCG@10 for the base and the context-aware ranking, averaged over 1000 random tie-breaks (seed 13).
- `harness/final_tables.py` writes the report tables in `eval/results/final/`.

Design decision. Scoring is a set of pure functions over saved raw output, so the metrics can be recomputed without re-scanning. **Limitation:** the ground truth is one rater's first pass, and it uses CAWASMA's own check ids, which favours CAWASMA's scope — stated openly in the write-up.

3 Numbers to know

All figures are from the repository at commit `7cc819f`.

3.1 The mapping in action

The same finding, scored on endpoints of each class, shows the mapping working.[5]

Check	base	LOW	MEDIUM	HIGH	CRITICAL
HDR-001 Missing HSTS	5.3	5.3	5.3	7.1	7.1
HDR-007 Server banner	5.3	5.3	5.3	6.1	6.1

A public page keeps the base score; a login or admin endpoint is raised.

3.2 Classifier accuracy, honestly

Sensitivity classification is a hard four-class problem on 164 labelled endpoints.[6]

Families	split	n	accuracy	macro F1
shipped (keywords)	test	113	0.425	0.362
proposed (anchors)	test	113	0.487	0.454
proposed (anchors)	dev	51	0.569	0.577

The anchor set beats the keyword set, but accuracy is modest. The project does not claim a strong classifier; it claims that a *reasonable* class, combined with body signals, moves scores in the right direction.

3.3 Detection and ranking

Detection (micro over four targets): CAWASMA recall 0.758, Nuclei 0.765, ZAP 0.333; all three precision 1.00.[7] Ranking (NDCG@10, environmental findings, base → context): mean 0.324 → 0.347; better than base only on DVWA (0.147 → 0.261).[8] The graded pools are not finished, so there is no pool-based NDCG yet.

[5] Source: `eval/results/final/mapping.md`
[6] Source: `eval/results/final/classifier.md`
[7] Source: `eval/results/final/tools.md`
[8] Source: `eval/results/final/ranking_environmental.md`

4 Contribution timeline

Grouped from merged pull requests.[9]

Merged	PRs	What changed
25 Oct	#8, #11	Set up the ML and evaluation workspace; evaluation setup.
25 Oct	#9, #18	Draft passive check specifications; check ids.
25 Oct	#10, #19	Environmental CVSS context mapping; mapping demo uses the real vectors.
25 Oct	#17	Pin `preshed` so spaCy loads; ML smoke test.
25 Oct	#20	Chains format 1.1: groups, scope, context, composite vectors.
27 Oct	#37	Evaluation ground truth.
27 Oct	#38	Response (body) classifier.
27 Oct	#40	Evaluation harness.
4 Nov	#49	Final evaluation, ranking experiment, tuning, security signatures.

Reviews received. Atta pushed fixes inside Asim's files on `atta/fix-asim-*` branches for Asim to review: catalog vectors (#58), the five check specs (#56), the sensitivity anchors (#60), the ranking experiment (#61), fake-secret values (#62), spec approval (#70), formatting (#71) and the refreshed pools (#72).

5 Viva preparation

5.1 Likely questions, with short answers

1. **Your ground truth uses your own check ids — isn't that biased?** Yes, and the write-up says so. It favours CAWASMA's scope, so ZAP and Nuclei are scored only on the checks their rules map to ("not covered" counts the rest).
2. **Why CVSS environmental metrics and not your own weighting?** Because the output is then a standard score anyone can reproduce; only the mapping to CR/IR/AR is ours, and it is a short, auditable table.
3. **Why "max" to combine class and signals?** It is conservative: one strong reason to care is enough to raise a requirement. Averaging would let a single public-looking token hide a real credential leak.
4. **Why does LOW map to X/X/X?** X is "Not Defined", the neutral baseline. A public endpoint should keep its base score; context raises scores, it does not lower public ones.
5. **The ranking isn't better overall — what does the project prove?** It proves the mechanism: on a sensitive, credential-bearing endpoint the same weakness scores higher, with a reproducible vector. The ranking study is honest that, on four targets, it is not better overall, and the pools are ungraded.
6. **Your classifier accuracy is under 0.5 — is that good enough?** For a fuzzy four-class problem it is modest but usable; the score moves in the right direction because body signals back up the class. We do not oversell it.
7. **How do you avoid flagging example data as a real secret?** The body classifier strips API-doc example blocks and filters placeholder values, and uses entropy to tell a real secret from a dummy.
8. **How is a credit-card number confirmed?** With a Luhn check; IBANs with an IBAN check. A number that fails the check is not reported as financial data.
9. **Why is the anchor classifier off by default?** To keep the default install light; the keyword classifier needs no model. The anchor set is the better, optional path.

[9] Source: `gh pr list (upstream); git log`

10. **How do you compare fairly with ZAP and Nuclei?** Each tool's rules are mapped to check ids; each is scored only on what it can report; the endpoint match is strict; false positives count only against explicit `absent` entries.
11. **Why did ZAP do so poorly?** It ran in passive baseline mode, which by design makes no attacks; these numbers do not represent a full active ZAP scan.
12. **What is NDCG@10 and why 1000 tie-breaks?** It scores how well a ranking puts graded items near the top; base scores tie a lot, so the result is averaged over 1000 random orderings of ties with a fixed seed.
13. **What are the five attack chains?** Session theft via XSS; error plus version banner; credentialed CORS read; credential interception on login; exposed config leaking credentials.
14. **How many ground-truth entries are there?** 33 present, 20 absent and 27 must-not-fire, across four targets — too few for a significance claim.
15. **Where does a check's base score come from?** The fixed CVSS v3.1 base vector in the catalog, not the catalog's `base_score` field (31 disagree; the vector wins).
16. **How does the engine keep working without your ML code?** Through the contract in `app/core/contracts.py`: the built-in keyword and regex classifiers are the fallback.
17. **Could the mapping be learned instead of hand-set?** In principle, with far more labelled data; today it is expert judgement, which is defensible and transparent.
18. **What stops a check specification from breaking the engine?** `make validate-config` checks every vector parses and matches `checks.yaml` before it can be used.
19. **How do you redact evidence?** `mask()` keeps only the last few characters; redaction also runs again on every export.
20. **What is the biggest threat to your evaluation's validity?** The single-rater ground truth and the small target count; both are stated as limitations.
21. **Why these four targets?** Juice Shop, DVWA, VAmPI and crAPI are well-known, intentionally vulnerable apps that between them cover web pages and REST APIs, so the ground truth can be written with confidence.
22. **Why keep a legacy formula at all?** Not every check has a reviewed spec yet; the 37 legacy checks keep working under the interim formula and migrate to the environmental path one at a time.
23. **How does a sensitivity class become a number?** Its CR/IR/AR triple is appended to the check's base vector and the `cvss` library computes the environmental score; nothing else changes.
24. **What does "not covered" mean in the comparison?** Ground-truth entries for checks a tool has no rule for; they are listed, not counted as misses, so the tool is judged only on what it can report.
25. **What would make the ranking result stronger?** Grading the priority pools (for a pool-based NDCG), more targets, and a second rater verifying the ground truth.
26. **How are abbreviations in a path handled?** `path_text()` can expand them (optionally with an LLM step) before the class is decided, so `/acct` and `/account` read alike.
27. **Is any of your scoring machine-learned?** The classifiers use embeddings, but the CR/IR/AR mapping is a fixed, human-set table — which is why the final score stays explainable.

5.2 Three-minute demo script

1. Open `cvss_mapping.yaml`. Say: "Class and signals become CR/IR/AR; the highest wins; LOW is X so public pages keep their base."
2. Show a HIGH finding's detail in the app: base 5.3, final 7.1, with the vector. Say: "Standard

CVSS environmental maths."

3. Run `make coverage-stats`. Point at 25 environmental vs 37 legacy.
4. Show `eval/results/final/tools.md` and `ranking_environmental.md`. Say the honest ranking result out loud.
5. Show `classifier.md`. Say: "Anchors beat keywords; accuracy is modest and we say so."

5.3 Five things to explain without notes

1. The mapping: class + signals → CR/IR/AR, combine by max, LOW=X.
2. Why a fixed base vector per check, and that only context is ours.
3. How the evaluation scores a tool only on the checks it can report.
4. The honest ranking and classifier numbers, and what they do and do not show.
5. The body classifier's placeholder/entropy filtering and redaction.

6 Glossary

CVSS base score
The standard severity of a vulnerability on its own, 0–10, from a vector such as `AV:N/AC:L/...`. Fixed per check; never invented per scan.

CVSS environmental score
The base score recomputed for a particular environment, using CR/IR/AR and modified base metrics. CAWASMA's context-aware score is this.

CR / IR / AR
Confidentiality, Integrity and Availability *Requirements*: environmental inputs saying how much each property matters here. Values Low, Medium, High, or X (Not Defined). CAWASMA sets them from the endpoint class and the data in the response.

Sensitivity class
CAWASMA's label for how sensitive an endpoint is — CRITICAL, HIGH, MEDIUM or LOW — from the words in its URL path.

Body signals
Sensitive data found in a response: credentials, personal data (PII), financial data, or an internal IP address. Evidence is redacted.

Base vs. final score
CAWASMA always keeps both: the official base score and the context-aware (environmental) final score, in separate columns.

Legacy vs. environmental scoring
A finding is scored either by the CVSS environmental metrics (approved specs) or by the interim legacy formula; its `scoring_method` says which.

Precision
Of the findings that were judged, the fraction that were correct: $\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$.

Recall
Of the real issues in the answer key, the fraction found: $\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$.

FPR
False-positive rate: $\mathrm{FP}/(\mathrm{FP}+\mathrm{TN})$.

Ground truth
A hand-written list, per target, of which check should (`present`) or should not (`absent`) fire on which endpoint.

NDCG@10
Normalised Discounted Cumulative Gain at rank 10: a score for how well a ranking puts the

important items near the top, against a graded list.

Tie-break sampling
Base scores tie often, so a ranking's NDCG is averaged over 1000 random orderings of the tied items, with a fixed seed.

SSRF
Server-Side Request Forgery: tricking the scanner into requesting an internal address. CAWASMA guards against it by validating the resolved IP on every hop.

Soft-404
A page that returns HTTP 200 but is really "not found". The crawler learns a baseline so it does not treat these as real endpoints.

Passive vs. active scan
Passive sends only safe GET/HEAD and reads what comes back; active sends extra, still non-destructive, probes. CAWASMA never sends attack payloads.

BOLA / IDOR
Broken Object-Level Authorisation / Insecure Direct Object Reference: reading another user's object by changing an id. CAWASMA only flags an *indicator*; confirming it needs logged-in accounts, which is future work.

Attack chain
Several findings that together make a bigger risk, correlated from a template and scored with their own composite vector.

Environmental metric "X"
"Not Defined": the neutral baseline. A public (LOW) endpoint uses X, so its score is unchanged — context only ever raises a score here, it does not lower a public page below its base.